

Employee Management System (Java)

Placement Training — Day 8 Project (D8) | SNPSU

GitHub Repo: **Project lot** | github.com/CoderS018/D4-Proj

Overview

This is a console-based Employee Management System written in Java. It lets a user create employee records, view all stored records, and give an employee a salary raise within a fixed percentage range, all through a simple numbered menu that loops until the user chooses to exit.

Code Structure

1. Employee class (inner static class)

A small data holder with four fields: **name**, **age**, **designation** and **salary**. Its constructor just assigns the four values passed in when a new employee is created. Every employee created by the program is stored as one **Employee** object inside an **ArrayList<Employee> employees**, which acts as the in-memory database for the whole session.

2. main() — the menu loop

main() creates one instance of the outer class (projd4 obj = new projd4()) so it can reuse the same Scanner and the same employees list across every menu choice. It then runs a do-while loop that keeps printing the four-option menu (Create, Display, Raise Salary, Exit) and reads the user's choice into an int. A Java 14+ switch expression (choice ->) routes to the matching method: create(), display() or raiseSalary(); choosing 4 just prints a goodbye message, and the loop condition (choice != 4) stops the program. Anything outside 1–4 falls into the default branch and prints an 'Invalid choice' message without crashing.

3. create() — adding employees

Runs inside a while(addMore) loop so the user can add several employees in one go. For each employee it:

- Reads the **name** as a full line (sc.nextLine()).
- Reads the **age** inside a do-while loop that keeps re-asking until a value above 18 is entered — this is input validation baked into the loop condition itself.
- Reads a one-letter **designation code** (P / T / M), uppercases it, and uses a switch expression to set both the designation text and a fixed base salary: Programmer → ■35,000, Tester → ■25,000, Manager → ■50,000. Any other letter falls back to 'Not Specified' with salary 0.
- Creates a new Employee object with these values and adds it to the employees list.
- Asks 'Add another? (Y/N)' inside its own validation loop that only accepts Y or N (case-insensitive), then sets addMore accordingly to continue or exit the outer loop.

4. display() — showing all records

If the list is empty it prints a friendly 'no records yet' message and returns early. Otherwise it loops over every Employee in the list with a normal for loop and prints an 'Employee #n' header followed by that employee's name, age, designation and salary.

5. raiseSalary() — giving a raise

- Also guards against an empty list first.
- Reads the employee's **name** to search for, then does a linear scan through the list using `String.equalsIgnoreCase()` so the search isn't case-sensitive. The first match becomes the target Employee object.
- If no employee matches the typed name, it prints a 'not found' message and returns.
- Otherwise it asks for a **raise percentage**, validated in a do-while loop to stay between 1 and 10.
- Calculates $\text{increment} = \text{salary} \times (\text{percent} / 100)$ and adds it directly onto `target.salary` — because `target` is a reference to the actual object stored in the `ArrayList`, this update is reflected immediately in the list without needing to put it back manually.

A few design notes

- Mixing `sc.nextInt()/sc.nextDouble()` with `sc.nextLine()` needs an extra `sc.nextLine()` call right after reading a number — the code does this consistently to flush the leftover newline, which is a very common Scanner bug in beginner Java programs.
- Validation is done with do-while loops that re-prompt on bad input rather than exceptions, keeping the program simple to follow (though it doesn't yet guard against non-numeric input for age/percentage, e.g. typing letters where a number is expected, since Scanner would throw an `InputMismatchException` there).
- Salaries are fixed per designation on creation, and `raiseSalary()` is the only place that changes them afterward.

Program Output (Terminal Run)



VS Code — Source & Terminal

Github Repo: [Project lot](#)

Github link: <https://github.com/CoderS018/D4-Proj>

Screenshots: Terminal

The screenshot shows an IDE with a project named 'proj4'. The main editor displays the 'proj4.java' file, which contains the following code:

```
116 public class proj4 {
117
118     void display() {
119         if (employees.isEmpty()) {
120             System.out.println("No employee records found. Please create one first.");
121             return;
122         }
123
124         System.out.println("----- employee details -----");
125         for (int i = 0; i < employees.size(); i++) {
126             Employee e = employees.get(i);
127             System.out.println("Employee " + (i + 1));
128             System.out.println("Name: " + e.name);
129             System.out.println("Age: " + e.age);
130             System.out.println("Designation: " + e.designation);
131             System.out.println("Salary: " + e.salary);
132         }
133     }
134
135     void raiseSalary() {
136         if (employees.isEmpty()) {
137             System.out.println("No employee records found. Please create one first.");
138             return;
139         }
140
141         sc.nextLine();
142         System.out.print("Enter the name of the employee to raise salary for: ");
143         String searchName = sc.nextLine();
144     }
145 }
```

The bottom panel shows the 'Terminal' tab with the following output:

```
PS C:\PLACEMENT TRAINING\proj4\proj4> java -cp ".\bin\classes" "com.example.proj4.Main"
----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice:
```

display() and raiseSalary() methods open in the editor alongside the live terminal.